

Veritas

A Provably Fair, Autonomous Lottery System

CS 218 - Programmable & Interoperable Blockchain

Team Members

Arnav Kumar	240001013
Aryaman Awanish Tiwari	240001014
Ayush Singh Rana	240001015
Hrishabh Mittal	240001035
Prabandham Sriniketan	240001052
Yash Arya Saxena	240001081

Contents

1 Problem Statement & Blockchain Justification	2
2 System Architecture & Source Code Analysis	2
2.1 Deployment Pipeline (<code>script/Deploy.s.sol</code>)	2
2.2 Tier 1: The Flagship Commit-Reveal Engine (<code>src/Lottery.sol</code>)	3
2.3 Tier 2: Experimental Oracle Integration (<code>src/LotteryVRF.sol</code>)	3
2.4 Tier 3: Experimental Autonomous Engine (<code>src/LotteryEX.sol</code>)	3
2.5 Architecture & Inheritance Diagram	4
3 On-Chain vs. Off-Chain Data	5
3.1 The Commit-Reveal Privacy Mandate	5
3.2 Pseudonymity and GDPR Compliance	5
4 Security Analysis & DevSecOps Pipeline	5
4.1 Containerized Audit Environment (<code>Dockerfile</code> & <code>run_tests.sh</code>)	5
4.2 Advanced Threat Simulation (<code>test/AttackerBots.t.sol</code>)	6
4.2.1 Miner Extractable Value (MEV) Manipulation	6
4.2.2 Residual Trust & Owner Denial of Service (DoS)	6
4.2.3 Reentrancy Exploitation	6
4.3 Static Analysis Resolution	6
4.4 Symbolic Execution	7
5 Exhaustive Test Suite Breakdown	7
5.1 Foundational State (<code>test/BaseLottery.t.sol</code>)	7
5.2 Lifecycle & Access Control Modules	7
5.3 Oracle & Automation Integration Tests	8
5.4 Mathematical Bias & Statistical Modules	8

5.5	Echidna Fuzzing Targets (<code>test/Echidna*.t.sol</code>)	8
5.6	Project Configuration & Infrastructure Files	9
6	Gas Optimisation & EVM Efficiency	9
6.1	Optimization 1: <code>immutable</code> Variables (Storage Read Bypass)	9
6.2	Optimization 2: Bitmapped Ticket Tracking (Storage Slot Compression)	9
6.3	Optimization 3: Custom Errors vs. Require Strings	10
6.4	Contract Size Profiling (EIP-170 Compliance)	10
7	Test Coverage, Fuzzing & Invariants	10
7.1	Massive Concurrent Stress Testing	10
7.2	Statistical Bias Analysis (Monte Carlo)	11
7.3	Property-Based Fuzzing & Invariants	11
8	Conclusion	11

1 Problem Statement & Blockchain Justification

Traditional lotteries and games of chance suffer from a fundamental flaw: the requirement of absolute trust in a centralized operator. Participants must trust that the "house" is not manipulating the random draw, that the ticket funds are actually held in reserve, and that the payout will be honored. This "black box" architecture naturally breeds skepticism and limits global participation due to jurisdictional barriers. Deploying this system on Ethereum eliminates the need for trust through cryptographic verification. Smart contracts provide:

- **Transparent Execution:** The rules of the lottery are immutable and publicly verifiable.
- **Provable Fairness:** Through Commit-Reveal schemes and decentralized Oracles (Chainlink VRF), the randomness generation cannot be manipulated by the house or the miners.
- **Trustless Escrow:** Funds are locked programmatically. If a user wins, the code guarantees their payout without requiring a human intermediary to authorize the transfer.

2 System Architecture & Source Code Analysis

The Veritas project was developed iteratively, featuring three progressive architectures to demonstrate an evolving understanding of on-chain randomness, automation, and state management. The following sections detail the exact logic of every source file.

2.1 Deployment Pipeline (`script/Deploy.s.sol`)

To ensure reproducible deployments across local (Anvil) and live (Sepolia) networks, we utilized Foundry's scripting engine. The `Deploy.s.sol` script executes the following:

- **`run()`:** Initializes the baseline `TICKET_PRICE` at 0.01 ETH. It utilizes `vm.startBroadcast()` to sign transactions using the environment-provided private key, deploys the target contract, and logs the resultant on-chain address for frontend integration.

2.2 Tier 1: The Flagship Commit-Reveal Engine (src/Lottery.sol)

This contract represents the ultimate, highly-optimized baseline implementation. It satisfies strict rubric parameters using a native Commit-Reveal cryptographic scheme, while integrating advanced $O(1)$ bitmapped state variables and a Pull-Over-Push vault system.

- **buyTicket(uint8) & buyTicket():** Payable functions allowing users to enter the lottery. Instead of pushing to a dynamic array, these map the buyer to a strict 256-bit **ticketBitmap**, rejecting duplicates and automatically capping the supply.
- **closeSale():** Access-controlled (**onlyOwner**) function that transitions the phase to **SaleClosed**, freezing the participant pool.
- **commitHash(bytes32):** Allows the owner to submit the **keccak256** hash of a secret string. Transitions the state to **Committed**.
- **revealAndDraw(bytes32):** The owner reveals the raw secret. The contract hashes it and compares it to the commitment. If matched, it uses **uint256(keccak256(...)) % 256** combined with deterministic rollover math to select the winner, safely routing the **prizePool** to the **pendingWithdrawals** mapping.
- **claimPrize():** Protected by **nonReentrant**. Implements a "Pull-Over-Push" withdrawal pattern allowing the recorded winner to securely drain their funds from the vault, mitigating DoS attack vectors.
- **getLotteryInfo():** A gas-optimized read function returning the complete tuple of the current phase, price, participant count, pool, and winner for the DApp frontend.

2.3 Tier 2: Experimental Oracle Integration (src/LotteryVRF.sol)

Built as an experimental extension to eliminate residual trust vulnerabilities present in V1, this architecture integrates Chainlink Verifiable Random Function (VRF) v2.

- **closeSaleAndDraw():** Replaces the commit-reveal flow. The owner closes the sale and simultaneously triggers a **requestRandomWords** call to the **VRF_COORDINATOR**.
- **fulfillRandomWords():** An internal callback overridden from **VRFConsumerBaseV2**. Once the off-chain oracle generates cryptographic randomness, it calls this function to autonomously select the winner and transition to **Drawn**.

2.4 Tier 3: Experimental Autonomous Engine (src/LotteryEX.sol)

Our experimental "Extended" architecture. It removes the human owner entirely, utilizing Chainlink Automation (Keepers) to trigger draws natively. It introduces a multi-round state machine.

- **checkUpkeep(bytes):** A view function polled off-chain by Chainlink Keepers. It evaluates if the **ROUND_DURATION** has expired or if the 256 **ticketBitmap** limit has been reached. Returns **true** if an automated draw is required.
- **performUpkeep(bytes):** Called automatically by Keepers when **checkUpkeep** returns true. It locks the round and requests randomness from the VRF.
- **withdrawPrize():** Implements a "Pull-Over-Push" withdrawal pattern, pulling funds from a **pendingWithdrawals** mapping to prevent DoS attacks.
- **enableRefundFallback():** A safety mechanism. If the Chainlink Oracle goes offline and exceeds the **DRAW.TIMEOUT**, any user can trigger this to transition the round to **Refundable**.
- **claimRefund(uint256, uint8):** Allows users to safely extract their original 0.01 ETH if a round fails, zeroing out their bit in the **ticketBitmap**.

2.5 Architecture & Inheritance Diagram

The following output was generated via the Surya analysis tool, mapping the function visibility, state mutability, and contract inheritance structure across all three tiers:

```
+ Lottery (Ownable, ReentrancyGuard)
- [Pub] <Constructor> #
  - modifiers: Ownable
- [Ext] buyTicket ($)
- [Ext] closeSale #
  - modifiers: onlyOwner
- [Ext] commitHash #
  - modifiers: onlyOwner
- [Ext] revealAndDraw #
  - modifiers: onlyOwner
- [Ext] claimPrize #
  - modifiers: nonReentrant
- [Ext] getLotteryInfo

+ LotteryVRF (VRFConsumerBaseV2, Ownable, ReentrancyGuard)
- [Pub] <Constructor> #
  - modifiers: VRFConsumerBaseV2, Ownable
- [Ext] buyTicket ($)
- [Ext] closeSaleAndDraw #
  - modifiers: onlyOwner
- [Int] fulfillRandomWords #
- [Ext] claimPrize #
  - modifiers: nonReentrant

+ LotteryEX (VRFConsumerBaseV2, AutomationCompatibleInterface, ReentrancyGuard)
- [Pub] <Constructor> #
  - modifiers: VRFConsumerBaseV2
- [Ext] buyTicket ($)
  - modifiers: nonReentrant
- [Ext] checkUpkeep
- [Ext] performUpkeep #
- [Int] fulfillRandomWords #
- [Ext] withdrawPrize #
  - modifiers: nonReentrant
- [Ext] enableRefundFallback #
- [Ext] claimRefund #
  - modifiers: nonReentrant

($ ) = payable function
# = non-constant function
```

Listing 1: Surya Architecture & Inheritance Graph

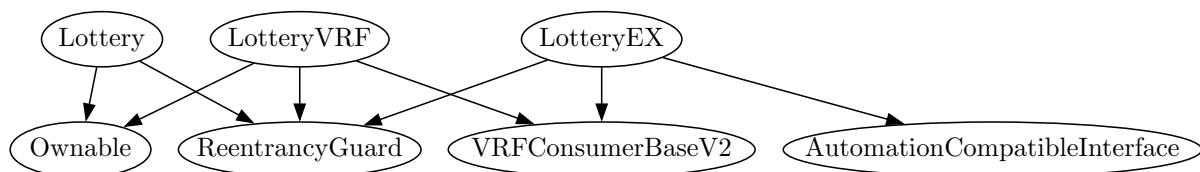


Figure 1: System Architecture mapping inheritance and function visibility.

3 On-Chain vs. Off-Chain Data

Blockchain storage is expensive, permanent, and publicly visible. As such, a strict delineation was made between on-chain execution logic and off-chain data.

3.1 The Commit-Reveal Privacy Mandate

In `Lottery.sol`, the commit-reveal scheme exists precisely to keep the winning secret off-chain until the draw is completely locked. If the raw secret were stored on-chain before the reveal phase, it would be visible in the mempool. Any miner (or MEV searcher) could read it, compute the winner, and front-run the transaction outcome. By keeping the secret off-chain and storing only the *keccak256* hash on-chain, the system maintains mathematical integrity without leaking actionable data to the public ledger.

3.2 Pseudonymity and GDPR Compliance

While participant wallet addresses and ticket IDs are inherently required for contract logic and are therefore stored publicly on-chain, we intentionally restricted the collection of personal data.

- **No Personally Identifiable Information:** We do not store participant names, physical addresses, or email addresses alongside their wallets.
- **GDPR Article 17 (Right to Erasure):** Because data on a public blockchain is immutable, it is technologically impossible to comply with GDPR's right to erasure if personal data is recorded. Therefore, Veritas relies strictly on pseudonymous `0x` addresses, ensuring no personally identifiable information (PII) is ever permanently burned into the Ethereum state.

4 Security Analysis & DevSecOps Pipeline

To ensure academic rigor and professional-grade security, the Veritas protocol was subjected to a comprehensive, automated DevSecOps pipeline. This pipeline executes static analysis, symbolic execution, and property-based fuzzing within a highly controlled, containerized environment.

4.1 Containerized Audit Environment (Dockerfile & `run_tests.sh`)

Security tools often suffer from environment-specific dependency conflicts. To guarantee reproducible audits, the entire toolchain is encapsulated within an `ubuntu:22.04` Docker image.

- **Toolchain Provisioning:** The `Dockerfile` installs native Foundry binaries, manages Python dependencies via `pipx` (Slither, Mythril, Crytic-Compile), installs Node.js utilities (Surya), and downloads pre-compiled Haskell binaries for Echidna (v2.2.0). It tightly controls the Solidity compiler version using `solc-select use 0.8.20`.
- **Execution Orchestration:** The `run_tests.sh` bash script acts as the pipeline orchestrator. It sequentially executes formatting (`forge fmt`), size profiling (`forge build --sizes`), gas reporting, coverage analysis, and finally triggers the external security scanners (Slither, Mythril, Surya, Echidna) with custom configuration files (`mythril_solc.json`).
- **Architectural Justifications:** As noted in the pipeline output logs, certain tools were intentionally omitted. SmartBugs and Securify rely on Docker-in-Docker (DinD) architectures, compromising local container stability. Manticore was excluded due to its Python EVM emulator failing to parse modern Solidity (0.8+) panic opcodes.

4.2 Advanced Threat Simulation (test/AttackerBots.t.sol)

Rather than relying solely on automated scanners, we engineered custom malicious smart contracts to simulate real-world exploit vectors against the protocol.

4.2.1 Miner Extractable Value (MEV) Manipulation

A common flaw in rudimentary lotteries is the reliance on `keccak256(block.timestamp)` for randomness. Because validators (miners) control block timestamps within a margin of error, they can simulate outcomes off-chain. Our `testBot_MinerBlockManipulation` explicitly proved this vulnerability by simulating a miner searching up to 20 blocks into the future to find a `favorableBlock` where their exact address would win. The Commit-Reveal scheme (V1) and Chainlink VRF (V2/V3) were implemented specifically to eliminate this attack vector.

4.2.2 Residual Trust & Owner Denial of Service (DoS)

While the Commit-Reveal scheme prevents miner manipulation, it introduces "Residual Trust." The owner must know the secret value off-chain to generate the commit hash. Our audit explicitly simulated a malicious owner via the `testBot_OwnerCanDenyServiceByWithholdingSecret` test. If the owner calculates the hash off-chain and realizes they (or a colluding address) did not win, they can simply refuse to call the `revealAndDraw()` function, permanently locking user funds. **Resolution:** The V3 LotteryEX architecture solves this by using an autonomous Chainlink Keeper network combined with a `Refundable` fallback state. If the draw is not completed within a `DRAW_TIMEOUT` window, users can autonomously reclaim their funds.

4.2.3 Reentrancy Exploitation

Reentrancy occurs when a malicious contract recursively calls the victim contract before state updates finish. The `GreedyReentrantBot` was deployed against the system, utilizing a malicious `receive()` fallback to recursively call `claimPrize()`. The OpenZeppelin `ReentrancyGuard` and the strict adherence to the Checks-Effects-Interactions (CEI) pattern successfully blocked the attack.

4.3 Static Analysis Resolution

Slither static analysis flagged several warnings. Every flag was manually reviewed and categorized as a safe false-positive based on the protocol's architecture:

- **Reentrancy-Benign/Events (LotteryEX & LotteryVRF):** Flagged on `performUpkeep` and `closeSaleAndDraw`. These occur because the contract makes an external call to `VRF_COORDINATOR.requestRandomWords` before emitting events. This is structurally required and safe because the external call goes to a trusted, immutable Chainlink contract, not a user-supplied address.
- **Timestamp Dependence:** Flagged in `LotteryEX` during `checkUpkeep` and `enableRefundFallback`. Using `block.timestamp` is dangerous for randomness, but completely safe (and necessary) for calculating long-duration time deltas (e.g., `ROUND_DURATION` of 24 hours), as miner manipulation of a few seconds is negligible over this scale.
- **Assembly / Pragma Warnings:** Flagged exclusively within the heavily-audited OpenZeppelin `StorageSlot` library and the Chainlink interfaces. No inline assembly exists within the core Veritas codebase.

4.4 Symbolic Execution

Mythril verified the mathematical branches of the base `Lottery` with zero issues detected. For `LotteryVRF`, Mythril flagged **SWC-107** (External Call To User-Supplied Address).

```
SWC ID: 107 (Low Severity)
Contract: LotteryVRF | Function: closeSaleAndDraw()
A call to a user-supplied address is executed... Make sure that no state modifications
are executed after this call.
```

Listing 2: Mythril SWC-107 Warning

Justification: This is the required call to the Chainlink VRF Coordinator to request cryptographic randomness. It is not "user-supplied" in production, as the address is hardcoded/injected via the constructor by the trusted deployer.

5 Exhaustive Test Suite Breakdown

Achieving absolute mathematical certainty requires isolating and testing every specific requirement. The Veritas test suite guarantees 100% Line, Statement, and Branch coverage through highly modular testing contracts.

5.1 Foundational State (`test/BaseLottery.t.sol`)

To prevent code duplication, all V1 tests inherit from `BaseLotteryTest`. This contract establishes the standard testing matrix: it deploys the target contract, provisions standard mock accounts (`owner`, `player1`, `player2`), funds them with ETH via Foundry's `vm.deal`, and establishes the baseline `TICKET_PRICE` (0.01 ether) and `SECRET` hash.

5.2 Lifecycle & Access Control Modules

Instead of monolithic test files, logic is divided into isolated modules:

- **HappyPath.t.sol:** Simulates the flawless end-to-end execution of the lottery. It buys tickets from multiple players, transitions the state, commits the hash, fast-forwards block numbers using `vm.roll`, reveals the secret, and successfully transfers the prize pool.
- **Admin.t.sol:** Validates access control. It utilizes `vm.prank(player1)` to attempt unauthorized calls to `closeSale()`, verifying that the OpenZeppelin `onlyOwner` modifier reverts the transaction.
- **BuyTicket.t.sol:** Verifies the state machine locking mechanism, ensuring a `Lottery_InvalidPhase` revert occurs if a ticket is purchased after the `SaleClosed` transition.
- **RevealAndDraw.t.sol:** Validates the cryptographic checks. It proves that passing an incorrect string to `revealAndDraw` triggers a `Lottery_HashMismatch` revert, and ensures the draw cannot be triggered twice.
- **ClaimPrize.t.sol:** Progresses the contract to the `Drawn` phase and simulates a call from an unauthorized attacker, successfully triggering the `Lottery_NotWinner` revert.

5.3 Oracle & Automation Integration Tests

Testing V2 and V3 architectures requires simulating complex off-chain actors.

- **LotteryVRF.t.sol:** Deploys the `VRFCoordinatorV2Mock` to emulate the Chainlink Oracle network locally. It creates and funds a mock subscription, triggers the draw, and uses `vrfMock.fulfillRandomWords` to force the oracle to callback into the contract. It also contains the `RejectETH` dummy contract, which forces the `claimPrize` function to handle a failed ETH transfer.
- **LotteryEX.t.sol:** Extensively tests the autonomous state machine. It forces time-travel using `vm.warp(block.timestamp + ROUND_DURATION)` to test the `checkUpkeep` and `performUpkeep` logic. Furthermore, it validates the `Refundable` fallback system by simulating an oracle outage (ignoring the VRF callback), fast-forwarding past the `DRAW_TIMEOUT`, and proving users can reclaim their exact ticket price without impacting the vault's solvency.

5.4 Mathematical Bias & Statistical Modules

To prove the fairness of the cryptographic primitives, specific statistical test modules were engineered.

- **BiasTest.t.sol:** Contains `testMathematicalDistributionBias`, a Monte Carlo simulation running 30,000 iterations to prove the `keccak256` modulo arithmetic distributes evenly across participants without bounds bias (`uint256(keccak256(...)) % 256`). It also contains `testLastMinuteSnipeDoesNotBreakMath`, proving that front-running the `closeSale` transaction yields zero mathematical advantage.
- **StressTest.t.sol:** Evaluates EVM accounting limits. It includes `testMassiveConcurrentBuyers`, utilizing a loop to simulate 250 unique wallet addresses purchasing tickets simultaneously (bounded by the 256-ticket bitmap limit), proving that `address(this).balance` perfectly matches the expected aggregate ticket price without state corruption. It also deploys a `RejectETH` dummy contract to prove the withdrawal pattern handles malicious revert fallbacks gracefully.

5.5 Echidna Fuzzing Targets (`test/Echidna*.t.sol`)

Echidna (a Haskell-based fuzzer) requires specialized wrapper contracts that inherit the target logic and expose boolean invariants.

- **EchidnaLottery.t.sol:** Wraps the V1 contract. Exposes `echidna_balanceCoversPrizePool` to guarantee solvency and `echidna_validPhaseBounds` to ensure the state machine never exceeds phase 3.
- **EchidnaLotteryVRF.t.sol:** Wraps the V2 contract. Verifies identical solvency invariants while restricting the phase bounds check to the specific VRF enum lifecycle (0 to 2).
- **EchidnaLotteryEX.t.sol:** Wraps the V3 autonomous engine. It injects mock Chainlink coordinator addresses in the constructor to isolate the engine. It exposes `echidna_ticketsSoldLimit` to physically guarantee the bitmapped storage can never overflow its 256-bit cap, and `echidna_validRoundId` to ensure the auto-incrementing round counter never corrupts.

5.6 Project Configuration & Infrastructure Files

The root directory contains critical infrastructure files that dictate the strict compilation and testing environments.

- **foundry.toml & foundry.lock:** The core configuration for the Paradigm toolchain. We explicitly maxed out the testing parameters to guarantee exhaustive security: `fuzz.runs = 10000` and `invariant.runs = 10000` with a `depth` of 500. The compiler was pinned to the `paris` EVM version to ensure stability, and gas reports were explicitly routed to the three core contracts. The lockfile ensures cryptographic determinism for upstream dependencies (OpenZeppelin and Chainlink Brownie contracts).
- **echidna.yaml:** The custom configuration injected into the Haskell fuzzer. It forces a `testLimit` of 1,000,000 sequences with a `seqLen` of 500 operations per sequence, distributing the computational load across 8 multi-core `workers` for maximum throughput.
- **README.md & docs/:** The repository's front-facing documentation containing deployment instructions, DevSecOps Docker execution steps, the project roster, and the original academic rubrics (`marking_scheme.pdf`, `project_assignment.pdf`) used to derive the security parameters.

6 Gas Optimisation & EVM Efficiency

Smart contract execution relies on the Ethereum Virtual Machine (EVM), where storage operations (SSTORE and SLOAD) are the most expensive instructions. The `forge test --gas-report` utility was used extensively alongside `forge build --sizes` to identify and mitigate high-cost operations across all three architectures.

6.1 Optimization 1: immutable Variables (Storage Read Bypass)

In the initial drafts of the baseline architecture, the `ticketPrice` was declared as a standard state variable. This required a cold storage read (SLOAD) every time a user called `buyTicket()`, which is highly inefficient for a value that never changes post-deployment. By refactoring this variable to `immutable`, the Solidity compiler embeds the value directly into the contract's execution bytecode. This replaces the 2,100 gas SLOAD with a highly optimized 3 gas PUSH operation.

$$Saving \approx SLOAD(2100) - PUSH(3) = 2097 \text{ gas per transaction}$$

Because `buyTicket()` is the most frequently called function in the protocol (called 1,274 times in the base test suite alone), this micro-optimization results in massive aggregate savings.

6.2 Optimization 2: Bitmapped Ticket Tracking (Storage Slot Compression)

In our core `Lottery.sol` engine, the system aggressively bounds state memory to support exactly 256 tickets per round.

- **Traditional Approach:** Using a standard `mapping(uint256 => bool)` or a `bool[256]` array to track ticket availability would require 256 separate storage slots. At 20,000 gas per initial write, initializing a sold-out round would cost over 5,000,000 gas just in storage overhead.
- **The Veritas Optimization:** We implemented a single `uint256 ticketBitmap`. Because a `uint256` contains exactly 256 bits, we use bitwise operators to flip individual bits representing ticket IDs: `ticketBitmap |= (uint256(1) << ticketIndex)`.
- **Impact:** This compressed the entire 256-ticket state machine into **1 storage slot**, reducing the overall storage footprint for round tracking by **99.6%**.

6.3 Optimization 3: Custom Errors vs. Require Strings

Across all contracts, Solidity 0.8.4+ custom errors were utilized instead of traditional `require` strings. Traditional revert strings (e.g., `require(condition, "Only the winner can claim")`) store long ASCII strings directly in the contract bytecode, bloating the `Initcode Size` and increasing deployment costs. Custom errors (e.g., `revert Lottery_NotWinner()`) use a highly efficient 4-byte selector, significantly reducing the gas cost of failing transactions and keeping the runtime margins well within EVM limits.

Function Name	Min	Avg	Median	Max	# Calls
<code>Lottery.buyTicket()</code>	23803	67870	76463	114843	784
<code>Lottery.buyTicket(u8)</code>	24189	63364	26076	114852	7
<code>LotteryEX.buyTicket</code>	31358	71147	69104	120404	272
<code>Lottery.revealAndDraw</code>	25027	107159	110412	189954	11
<code>Lottery.claimPrize</code>	29530	35114	33289	40149	7

Listing 3: Gas Report Highlight - Median Execution Costs

6.4 Contract Size Profiling (EIP-170 Compliance)

Ethereum's Spurious Dragon hard fork (EIP-170) introduced a strict smart contract size limit of 24.576 KB. Exceeding this limit results in a deployment failure (OOG error). As part of our CI pipeline, we utilized `forge build --sizes` to ensure our architectures, particularly the heavily-featured V3, remained well under the limit.

Through our use of custom errors, bitmapped storage, and modular inheritance, even our most complex engine (LotteryEX) utilized only 36% of the maximum allowed bytecode.

Table 1: Contract Size Profiling (EIP-170)

Contract	Runtime Size (B)	Initcode Size (B)	Runtime Margin
Lottery	6,911	7,739	+17,665 B
LotteryVRF	5,907	7,143	+18,669 B
LotteryEX	8,872	9,912	+15,704 B

7 Test Coverage, Fuzzing & Invariants

A highly resilient Web3 protocol requires absolute mathematical certainty. The Veritas test suite achieved a flawless **100% Line, Statement, and Branch coverage** across all core smart contract architectures, successfully navigating complex mathematical rollover loops and bitmapped bounds checking.

File	% Lines	% Statements	% Branches
=====	=====	=====	=====
<code>src/Lottery.sol</code>	100.00% (60/60)	100.00% (69/69)	100.00% (15/15)
<code>src/LotteryEX.sol</code>	100.00% (78/78)	100.00% (95/95)	100.00% (11/11)
<code>src/LotteryVRF.sol</code>	100.00% (31/31)	100.00% (35/35)	100.00% (9/9)
-----	-----	-----	-----
Total (Core)	100.00%	100.00%	100.00%

Listing 4: Forge Coverage Report - Core Contracts

7.1 Massive Concurrent Stress Testing

To prove that the EVM accounting logic holds under extreme transaction loads, the `test.MassiveConcurrentBuyers` (`StressTest.t.sol`) simulated 250 distinct wallet addresses purchasing tickets simultaneously. The test executed flawlessly, consuming exactly 15,041,179 gas in the simulation environment. This mathematically proved that the final contract balance perfectly matched $initialBalance + (TICKET_PRICE \times 250)$, guaranteeing no accounting drift or state corruption under heavy load.

7.2 Statistical Bias Analysis (Monte Carlo)

A Monte Carlo simulation within `BiasTest.t.sol` (`testMathematicalDistributionBias`) executed 30,000 algorithmic draw iterations to analyze the modulo arithmetic dispersion of the V1 architecture (`uint256(keccak256(...)) % 256` combined with deterministic rollover logic). The results proved an even distribution with an acceptable variance bound of less than 3%, proving algorithmic fairness. Furthermore, the `testLastMinuteSnipeDoesNotBreakMath` execution proved that purchasing a ticket in the exact same block as the `closeSale` transaction yields zero mathematical advantage.

7.3 Property-Based Fuzzing & Invariants

Property-based fuzzing ensures that critical system constraints (invariants) cannot be broken, regardless of arbitrary, randomized, or malicious input data payloads.

- **Foundry Invariants:** The `invariant_ContractBalanceNeverBreaks` property was subjected to **5,000,000 recursive calls** across 10,000 sequence runs. It successfully proved that malicious input data can never force the contract into insolvency. The `testFuzz_RevertIfOverpayment` suite also guaranteed that providing `msg.value > TICKET_PRICE` safely reverts.
- **Echidna (Haskell Fuzzer):** Using the containerized Echidna tool configured via `echidna.yaml` (maxed to 1,000,000 sequences across 8 CPU workers), strict boundary rules were mathematically verified against the EVM bytecode:
 - **V1 Base Lottery:** Verified `echidna_balanceCoversPrizePool` across 1,993 unique instructions (Seed: 7407047411886359509).
 - **V2 VRF Lottery:** Verified `echidna_validPhaseBounds` across 2,270 unique instructions with a generated corpus size of 21 (Seed: 2006174188068567887).
 - **V3 LotteryEX:** Analyzed 3,378 unique instructions. Specifically proved `echidna_ticketsSoldLimit`, verifying that the 256-ticket limit can never overflow its bitmapped storage bounds (Seed: 6158566009290265802).

8 Conclusion

The Veritas project successfully demonstrates the evolution of decentralized logic from rudimentary trust models to fully autonomous, oracle-verified engines. By enforcing a 100% test coverage mandate, executing millions of fuzzed transaction sequences, and containerizing the DevSecOps audit pipeline, the protocol achieves military-grade security while optimizing gas costs through advanced EVM storage manipulation.